

LevelSetWear user manual

Abaqus user elements for wear simulation on a fixed mesh

Iñigo Llavori

Mondragon Unibertsitatea, Faculty of Engineering, Department of Mechanical Engineering, Loramendi 4, 20500 Arrasate-Mondragon, Spain

Abstract

This manual accompanies `uelWearLS.for`, a set of user elements (UEL) for Abaqus/Standard that simulate wear on a fixed finite element mesh. The worn surfaces of two bodies in contact are level sets, the elements cut by them are integrated with fixed points, and the contact between the worn surfaces is enforced by an immersed contact element. Since the mesh and its integration points never move, the variables stored at them stay with the material while the surfaces wear. Five examples are provided, a plate with a hole, frictional contact, block-on-ring and fretting wear, and the level set update. The files can be downloaded from <https://github.com/illavori/LevelSetWear>. If using this code for research or industrial purposes, please cite

I. Llavori, Wear simulation on a fixed mesh with level sets and immersed frictional contact, submitted for publication.

This manual describes the essential features of the model and gives the instructions to use the subroutines, in version 5 of the code, the one used in the article. The theory and its verification are given in the article.

Keywords: Wear, Level set, Fixed mesh, Immersed contact, Fretting, Abaqus

1. The level set wear model

Finite element wear models usually move the nodes of the mesh to follow the worn surface, for instance with the `UMESHMOTION` subroutine of Abaqus.

Email address: `illavori@mondragon.edu` (Iñigo Llavori)

The integration points then move with the nodes and no longer coincide with the material they represented, so any variable with history has to be transferred to the new positions after every wear update. The model implemented here does not move the mesh. Its four ingredients, Sections 3.1 to 3.4 of the article, are summarised below in the order in which a wear increment uses them.

Worn surfaces as level sets. The surface of each body is the zero contour of a level set function $\phi^{(b)}$, $b = 1, 2$, a signed distance to the surface that is negative in the material and positive in the void. It is stored on a fixed Cartesian grid much finer than the finite element mesh, and only in a narrow band of a few cells around the surface. Body 1 is the flat (slave) and body 2 the cylinder (master).

Cut elements with fixed integration points. An element crossed by a surface is integrated only over its material part. Its $n \times n$ Gauss points are fixed. The parent square is split into cells, one around each point, whose sides are the Gauss weights, and the weight of a point is the material area of its cell, the cell rule. As the surface crosses the element, the weight of each point falls continuously from its full value to zero, so the stiffness of the element decreases smoothly and a point keeps its position, and its history, while it has material around it. Elements in the void keep a small fraction ε_v of their stiffness.

Immersed frictional contact between level set surfaces. Contact acts between the straight segments of the surfaces inside the cut elements. One contact element is defined for every interior vertex of the slave surface. It samples the gap g at three Gauss points on each of the two segments that meet at the vertex, corrected with the level sets for the faceting of the segments, and gathers them into a single weighted gap of the vertex,

$$\tilde{g}_v = \frac{1}{D_v} \int \Phi_v g \, ds, \quad D_v = \int N_v \, ds, \quad \Phi_v = 3N_v - 1, \quad (1)$$

where N_v is the hat function of the vertex and Φ_v its dual function (dual mortar). A penalty law gives the pressure of the vertex, $p_v = \varepsilon_N \max(-\tilde{g}_v, 0)$, and a Coulomb law with a return mapping its shear, with the slip weighted in the same way.

Wear update. At the end of every converged increment the local Archard law gives the wear depth of each body at every contact point,

$$\Delta h^{(b)} = \Delta N k_b p \Delta s, \quad (2)$$

with k_b the wear coefficient of the body, Δs the slip of the increment and ΔN the number of real cycles that each simulated cycle stands for (cycle jump). The depths are carried to the zero contour with a kernel of radius ℓ_e , the element size, and extended to the narrow band along the normals. The level set is rebuilt as the signed distance to the current surface plus the extended depth, which moves the zero contour into the material and reinitialises $\phi^{(b)}$ at every update (Section 3.4 of the article). The mesh and the integration points are not touched. The update is explicit, so the wear of an increment must stay well below the element size.

2. The Abaqus subroutines

The file `src/uelWearLS.for` contains four subroutines that work together. Two element types are defined with the `UEL` subroutine, the cut bulk element `U1` and the contact element `U2`. A `UMAT` defines a ghost mesh used only to visualise the results of `U1` in Abaqus/Viewer. The `UEXTERNALDB` subroutine performs the wear update at the end of every converged increment and writes the results at the end of every step. The elements share data through `COMMON` blocks, so the element operations must run with one processor (`cpus=1`). Only the equation solver can run in parallel, with `standard_parallel = SOLVER` and `mp_mode = THREADS` in the `abaqus_v6.env` file of the job folder. This gives identical results, but no speed-up for the models of the examples, whose time is spent mostly outside the solver. Units are mm, N and MPa, the analysis is two-dimensional in plane strain, and the file paths are built for Windows. The code needs Abaqus/Standard with a Fortran compiler configured for user subroutines, and it has been tested with Abaqus 2020 and Intel Fortran 2021 on Windows.

2.1. Cut bulk element *U1*

`U1` is a four-node plane strain quadrilateral with two degrees of freedom per node, identical to the Abaqus `CPE4` element with its B-bar treatment of the volumetric strain when it is not cut. Its properties are listed in Table 1. The elements of the wear box of the flat and of the cylinder form two element

Table 1: Properties of the cut bulk element U1 (*UEL PROPERTY).

PROPS	Meaning	Value in the examples
1	Young's modulus E [MPa]	200000 or 210000
2	Poisson's ratio ν	0.3
3	SIDE, sign of the level set, $\phi = \text{SIDE}(y - y_s(x))$	+1 flat, -1 cylinder
4	body IB (surface table and grid)	1 flat, 2 cylinder
5	void stiffness factor ε_v	10^{-6}
6	NFIX, fixed $n \times n$ rule (0 gives a moving polygon rule)	4

sets, with the side and the body of their level set given by properties 3 and 4.

The solution-dependent variables of U1 are the stresses averaged over the material part of the element (SVARS 1 to 4, σ_{11} , σ_{22} , σ_{33} , σ_{12} in MPa), its material volume fraction (SVARS 5) and the material fraction of the cell of each fixed point (SVARS $5 + k$, $k = 1 \dots n^2$). The element is declared as

```
*USER ELEMENT, NODES=4, TYPE=U1, PROPERTIES=6, COORDINATES=2,
  VARIABLES=21
1, 2
*ELEMENT, TYPE=U1, ELSET=UELF
... connectivity of the flat wear box ...
*UEL PROPERTY, ELSET=UELF
200000, 0.3, 1., 1., 1e-06, 4
```

with $\text{VARIABLES} = 5 + n^2$ and an equivalent block, with $\text{ELSET}=\text{UELC}$ and properties -1., 2. in positions 3 and 4, for the cylinder.

2.2. Immersed contact element U2

U2 is defined for every interior vertex of the slave surface. Its nodes are three columns of slave nodes, around the vertex, and a window of M columns of master nodes, each column with $K + 1$ nodes from the surface inwards, so that the element sees the surfaces while they wear through K rows of elements. Its properties are listed in Table 2. The first six define the contact, properties 7 to 11 the wear and property 12 an imposed sliding, used when only the normal contact is solved, as in continuous sliding. Properties 7 to 12 are optional. With six properties the element solves contact without wear, as in example 02.

Table 2: Properties of the contact element U2 (*UEL PROPERTY).

PROPS	Meaning	Value in the examples
1	normal penalty ε_N [MPa/mm]	$10E/\ell_e$
2	tangential penalty ε_T [MPa/mm]	$\varepsilon_N/10$ (03, 04), ε_N (02)
3	friction coefficient μ	0.6 (02), 0 (03), 0.9 (04)
4	rows of elements K seen by the element	3 (02, 04), 4 (03)
5	master node columns M	9
6	label offset, the label of the first contact element minus one	200000 (02), 90000 (03), 300000 (04)
7	wear coefficient k_1 of the flat [MPa $^{-1}$]	2×10^{-9} (03), 2.9×10^{-8} (04)
8	wear coefficient k_2 of the cylinder [MPa $^{-1}$]	0 (03), 1.9×10^{-8} (04)
9	cycle jump ΔN	1 (03), 100 (04)
10	first wear step	3 (03), 4 (04)
11	warning threshold of the surface change per increment [mm]	10^{-3}
12	imposed sliding per unit step time [mm] (optional)	80 (03)

The 24 solution-dependent variables of U2 are the position of the vertex, its pressure, shear, accumulated slip, gap and state (0 open, 1 stick, 2 slip) in SVARS 1 to 6, followed by the shear, slip and pressure of each of its six Gauss points. The element is declared as

```
*USER ELEMENT, NODES=48, TYPE=U2, PROPERTIES=11, COORDINATES=2,
  VARIABLES=24, UNSYMM
1, 2
*ELEMENT, TYPE=U2, ELSET=CONTACT
... one element per slave vertex ...
*UEL PROPERTY, ELSET=CONTACT
2000000000, 200000000, 0.9, 3, 9, 300000, 2.9e-08, 1.9e-08,
100, 4, 0.001
```

where the number of nodes is $(M + 3)(K + 1)$.

2.3. Visualisation

Abaqus/Viewer cannot display the results of user elements. As usual, a ghost mesh of standard CPE4 elements is placed on the same nodes as the U1 elements, with a negligible stiffness (10^{-8} MPa) defined by the UMAT, which copies the results of each U1 element to the solution-dependent variables of its ghost element (SDV1 to SDV4 the averaged stresses, SDV5 the material

volume fraction). The ghost elements are labelled with the U1 label plus an offset, given to the UMAT as its only constant,

```
*ELEMENT, TYPE=CPE4, ELSET=GHOST
*SOLID SECTION, ELSET=GHOST, MATERIAL=GHOST
*MATERIAL, NAME=GHOST
*USER MATERIAL, CONSTANTS=1
120000
*DEPVAR
6
```

DEPVAR is 5 or 6. SDV6 is reserved for the level set, which is not written to the ghost mesh in this version and stays at zero. The worn surfaces are written to `wearProfile.txt` (Section 2.4).

2.4. Files read and written

The subroutines read the following files from the folder of the job.

- `surf1.txt` and `surf2.txt` (required). Initial surface of the flat and of the cylinder, as a first line `N NSUB` followed by `N` lines `x y` with increasing x . Every `NSUB`-th point is a surface node of the mesh, and the other points describe the true shape between them.
- `lsgrid.txt` (optional). If present, each surface is a narrow band level set on a Cartesian grid, the mode used in all wear analyses. One line per body, `X0 X1 Y0 Y1 DX NBAND NREINIT HKER`, with the grid box and spacing [mm], the half width of the band in cells, the number of updates between redistancing and the radius of the wear kernel [mm]. The box must contain all the wear. The examples use $DX = \ell_e/16$, $NBAND = 6$, $NREINIT = 1$ and $HKER = \ell_e$. Only $NREINIT = 1$, the value of the article, has been verified. Without the file the surfaces are the tables themselves, used for the contact verification.
- `v2opts.txt` (optional). One option per line. `MORTAR 2` selects the dual mortar contact used in the article, and `WSM 1` averages the wear of the contact points over their vertex, which `MORTAR 2` already does by itself. Without the file the contact law acts independently at every Gauss point.

At the end of every step they write `contactOut.txt`, with one line per contact element (`IC x p tau slip gap status`), and `wearProfile.txt`, with

the surface of each body at the surface nodes (x y). Each step starts with a line `STEP n TIME t`, and each body in `wearProfile.txt` with a line `BODY ib n`. The `.msg` file reports the version, the options and the time spent in each subroutine.

3. Usage

The simplest way to run the model is to start from one of the examples, whose input files contain every block described below. They were generated with a script from a conventional Abaqus model of the two bodies in contact. A new model can be built in the same way, following the steps below, or by adapting the input files of the examples.

The first step is to create the model in Abaqus/CAE as usual, with three differences.

- Both bodies are meshed with CPE4 elements, and the region that the surfaces will cross, the wear box of each body, with a structured mesh of square elements of size ℓ_e , at least K rows deep.
- No contact is defined between the two bodies, since the contact elements take its place.
- Each wear block, one simulated cycle standing for ΔN cycles or a block of sliding distance, is a step. Wear starts at the step given by property 10 of U2, so the steps that apply the loads come first.

Once the model has been developed, the input file is written and modified with a text editor.

1. The elements of the wear box of the flat are defined as U1 elements in the set `UELF`, and those of the cylinder in the set `UELC`, with the `*USER ELEMENT` and `*UEL PROPERTY` blocks of Section 2. Their connectivity is that of the original CPE4 elements.
2. One U2 element is added for every interior vertex of the contact surface of the flat. Its slave nodes are the three columns of nodes of the flat around the vertex and its master nodes the M columns of nodes of the cylinder closest to it, each column with $K + 1$ nodes from the surface inwards. The slave node in row r ($r = 0$ at the surface) and column $s = 0, 1, 2$ (the vertex in column 1) is local node $3r + s + 1$, and the master node in row r and column $c = 0 \dots M - 1$ is local node $3(K + 1) + c(K + 1) + r + 1$.

3. The ghost CPE4 elements are added on the nodes of the U1 elements, with the **UMAT** material of Section 2 and **SDV** requested as field output of the set **GHOST**.
4. The files **surf1.txt**, **surf2.txt**, **lsgrid.txt** and **v2opts.txt** of Section 2.4 are written in the folder of the job.

The job is then run in the same folder with

```
abaqus job=case user=uelWearLS.for cpus=1 interactive
```

A few rules keep the analysis stable. The worn surface must stay within the K rows seen by the contact element, and a warning in the **.msg** file reports otherwise. The wear of an increment must be small compared with the element size. As a rule of thumb for continuous sliding, with E^* the contact modulus of the two bodies, a stable sliding increment is $\Delta s < 2L/(\pi k_b E^*)$, with $L = 2\ell_e$. The hard limits of the arrays (for instance 200 000 for the largest U1 label, 5000 contact elements and a level set grid of at most 8001×241 nodes) are listed in the header of the Fortran file.

4. Examples

Each example is a folder ready to run, with the input file, the auxiliary files and a **run.bat** (replace **abq2020** by the command of your Abaqus installation), and the results of the article in a **reference** folder where available. Every result of the article can be reproduced. Each example holds the input files of all its analyses and the scripts that extract their results into the CSV files of the folder **figures** of the repository, where **python plot.py** in each subfolder draws the figure of the article. Each script has its usage in its header and prints the numbers of the tables of the article. On the runs of the article the scripts give the CSV files of the repository bit for bit. Scripts marked Abaqus Python are run with **abaqus python**, the others with Python 3 and NumPy.

01, plate with a hole. A quarter of a plate with a circular hole under remote tension, on a 64×64 grid that ignores the hole (Section 4.1 and Fig. 9 of the article, which shows the 256×256 grid). The hole is a level set, and the elements it cuts are integrated with the cell rule. This example uses the bulk element of **uelWearBulk.for**, which accepts a circular level set, given in **surf1.txt** by a first line 2 -1 followed by the centre of the hole and its radius. **makeKirsch.py** writes the 21 inputs of the article, seven grids

from 16×16 to 1024×1024 with the cell rule, the exact integration of the material polygon and the point rule. `extractKirsch.py` (Abaqus Python) reads the displacements and the stresses of the cut elements from each odb, and `postKirsch.py` computes the strain energy U_h from the displacements, the energy norm error $\sqrt{(U - U_h)/U}$, with the exact U in the header of `reference/kirsch.txt`, and the error of the cut elements, and writes the data of Fig. 9. The steps are in the `README.txt` of the example. The plate with a hole is a benchmark of fixed-grid and cut element methods [1].

02, Mindlin contact. A cylinder of radius 6 mm on a flat, loaded normally (120 N/mm), tangentially to $Q^* = 36$ N/mm and unloaded, with $10 \mu\text{m}$ elements, $\mu = 0.6$ and $\varepsilon_T = \varepsilon_N$ (Section 4.2 and Figs. 11 and 12 of the article). In `aligned` the surfaces lie on the nodes, and in `immersed` they are immersed at $\Delta y_F = \Delta y_C = \ell_e/2$ in their first row of elements. `standard_abaqus` is the same model with the standard contact of Abaqus, without a reference folder. The surface pressure and shear of `contactOut.txt` are compared with the solutions of Mindlin–Cattaneo (step TANGENTIAL) and Mindlin–Deresiewicz (step UNLOAD) [2], and with `reference/contactOut.txt` or `reference/contactOut_imm_F50C50.txt`. `immersed/makeImmersed.py` writes the seven immersed cases of Table 2 of the article, whose results are also in `immersed/reference`. `extractOdb.py` (Abaqus Python) reads the odb of the level set and of the standard models, and `mindlinResults.py` writes the data of Figs. 11 and 12 and prints Tables 1 and 2.

03, block-on-ring. The block-on-ring test of Cruzado et al. [3] (Section 5.1 and Fig. 15 of the article), 10.8 N/mm without friction, 3600 m of sliding in 45 wear blocks of 80 m, with an imposed sliding of 80 mm per increment and $20 \mu\text{m}$ elements. Only the block wears, with $k_1 = 2 \times 10^{-9} \text{ MPa}^{-1}$. The worn profile of the block in `wearProfile.txt` is compared with `reference/wearProfile.txt`. The analysis takes about 18.5 h of CPU time. `extractBOR.py` writes the data of Fig. 15 and prints the numbers of Table 3, from a run or, with the argument `reference`, from the shipped results. The folder `mesh_motion` holds the mesh-motion model of the comparison, with the subroutine `src/umeshArchard2D.for` (about 25 h of CPU time). `odbSurfaces.py` (Abaqus Python) reads its worn surfaces from the odb and `extractMeshMotion.py` writes its data of Fig. 15 and Table 3.

04, fretting. The fretting test of McColl et al. [4], a cylinder of radius 6 mm on a flat under 18.5 N/mm with a stroke of $\pm 25 \mu\text{m}$, 18 000 cycles in 180

wear blocks of 100 cycles, with both bodies wearing (Section 5.2 and Figs. 16 and 17 of the article). The worn profiles of both bodies are compared with `reference/wearProfile.txt`. The analysis takes about 5.7 h of CPU time. `extractMcColl.py` writes the data of Figs. 16 and 17 and prints the numbers of Table 3, from a run or from the shipped results (argument `reference`). The mesh-motion model of the comparison is in `mesh_motion` (about 4.7 h of CPU time), where `extractMeshMotion.py` writes its data of Figs. 16 and 17 and Table 3.

05, level set update. The narrow band update of the level set is compared with the exact solution of a height function, on a self-consistent wear model of a rigid cylinder of radius 6 mm sliding on a flat (Winkler foundation, Archard law), for 18 000 cycles (Section 4.3 and Fig. 13 of the article). The two Python files do not need Abaqus. `protoNBLS.py` contains both representations of the surface, the level set with the same four operations as the subroutine, and `paperLS.py` runs them and draws the figure of the article (`levelsetUpdate.pdf`, to be compared with `reference/levelsetUpdate.pdf`). It needs NumPy, SciPy and Matplotlib.

5. Limitations

The present version is two-dimensional, in plane strain, with four-node quadrilateral elements, a linear elastic material, a flat (body 1) and a cylinder (body 2), and small sliding within each increment. The wear update is explicit, so the increment must be kept small. The elements must run on one processor, since they share data through `COMMON` blocks. For three-dimensional models, whose element operations are far more expensive, the code would have to be reorganised so that each element keeps its data in its own variables and the elements can run truly in parallel. The level set is not written to the ghost mesh. These limitations come from the implementation, not from the method. The cell rule and the immersed contact apply equally to other element types, such as triangles or higher-order elements, and the extension to three dimensions and to history-dependent materials is the subject of ongoing work.

References

- [1] N. Sukumar, D. Chopp, N. Moës, T. Belytschko, Modeling holes and inclusions by level sets in the extended finite-element method, *Computer Methods in Applied Mechanics and Engineering* 190 (2001) 6183–6200.
- [2] K. L. Johnson, *Contact mechanics*, Cambridge University Press, Cambridge, 1985.
- [3] A. Cruzado, A. Zabala, M. A. Urchegui, X. Gómez, Definición de una metodología optimizada para la simulación del desgaste en materiales metálicos, *Revista de Metalurgia* 46 (Extra) (2010) 106–114.
- [4] I. R. McColl, J. Ding, S. B. Leen, Finite element simulation and experimental validation of fretting wear, *Wear* 256 (2004) 1114–1127.